CPNCheck

Model checking de Redes de Petri Coloridas

fundamentos, implementação e por que confiar no resultado

Documentação técnica

5 de outubro de 2026

Sobre este documento. Ele começa do zero: o que é model checking, para que serve, e por que a resposta que ele dá é diferente da resposta que um teste dá. Depois desce até a implementação do CPNCheck módulo por módulo, e termina com a parte que mais importa para quem vai defender um resultado — o que exatamente está garantido, o que não está, e quais evidências sustentam cada afirmação.

Não é preciso conhecer lógica temporal nem Redes de Petri para começar. Os capítulos 2 e 3 constroem os dois conceitos do início.

Sumário

1 O problema **3**
1.1 Um teste responde uma pergunta menor do que parece . . . 3
1.2 O que é preciso para verificar . . . 3

2 Model checking do começo **5**
2.1 O modelo: sistemas de transição . . . 5
2.2 A propriedade: lógica temporal CTL . . . 5
2.3 O algoritmo: pontos fixos . . . 6
2.4 O ponto delicado: marcações mortas . . . 7
2.5 O limite real: explosão de estados . . . 7

3 Redes de Petri Coloridas **9**
3.1 Rede de Petri . . . 9
3.2 O que "colorida" acrescenta . . . 9
3.3 Hierarquia . . . 10
3.4 O grafo de ocorrências . . . 10
3.5 Três conceitos derivados do grafo . . . 11

4 Para que serve isso num modelo de *machine learning* **12**
4.1 O que se está verificando — e o que não se está . . . 12
4.2 A escada de escopo . . . 12

5 CPNCheck por dentro **14**
5.1 Arquitetura . . . 14
5.2 Camada por camada . . . 14
5.2.1 `net.py` — ler o arquivo . . . 14
5.2.2 `cpnml.py` — entender as expressões . . . 14
5.2.3 `colourset.py` — os domínios . . . 15
5.2.4 `coloured.py` — a rede que dispara . . . 15
5.2.5 `statespace.py` — o grafo . . . 16
5.2.6 `ctl.py` — os pontos fixos . . . 16
5.2.7 `formula.py` — propriedades como dados . . . 16
5.2.8 `profile.py` e `profiles/` — onde entra o domínio . . . 16
5.3 As quatro técnicas . . . 17
5.4 Exatidão da análise estrutural . . . 17
5.5 O catálogo de propriedades . . . 18

6 Contraexemplos **19**
6.1 Por que um veredicto sozinho não basta . . . 19
6.2 As cinco formas . . . 19

6.3 O laço, e por que ele precisou de um algoritmo . . . 19
6.4 O certificado: `replay()` . . . 20
6.5 De achado estrutural para entrada concreta . . . 20

7 Por que confiar no resultado **21**
7.1 O problema de fundo . . . 21
7.2 Primeira evidência: sensibilidade (mutantes) . . . 21
7.3 Segunda evidência: concordância com o CPN Tools . . . 22
7.4 Terceira evidência: os contraexemplos redisparam . . . 22
7.5 O que *não* está garantido . . . 22

8 Uso prático **24**
8.1 Linha de comando . . . 24
8.2 Python . . . 24
8.3 Como ler o relatório . . . 24
8.4 Verificando redes que não são de *machine learning* . . . 25

Glossário **26**

Capítulo 1

O problema

1.1 Um teste responde uma pergunta menor do que parece

Suponha que você exportou um modelo de *machine learning* — uma árvore de decisão, uma floresta aleatória, um *gradient boosting* — para uma Rede de Petri Colorida, e quer ter certeza de que a rede se comporta bem. A primeira coisa que ocorre é testar: rodar a rede com mil entradas e conferir mil saídas.

Isso é valioso, mas responde a uma pergunta estreita: *"para estas mil entradas, a rede deu a resposta certa?"*. Não responde:

- a rede sempre *para*, ou existe entrada que a faz girar para sempre?
- existe algum estado em que ela trava sem produzir resposta?
- existem duas folhas da mesma árvore que podem estar habilitadas ao mesmo tempo — ou seja, a rede é realmente determinística?
- existe alguma execução em que o resultado é depositado duas vezes?

Nenhuma dessas perguntas é sobre uma entrada específica. Todas são sobre *todos os comportamentos possíveis* do modelo. Teste é amostragem: executa alguns caminhos e conclui por indução informal. Model checking é exaustão: enumera *todos* os estados alcançáveis e decide a pergunta sobre o conjunto inteiro.

A distinção que vale memorizar. Um teste que passa diz "não encontrei problema nestes casos". Um model checker que responde "verdadeiro" diz "não existe problema, ponto" — dentro do modelo e do escopo que ele explorou, que é exatamente o que este documento vai tornar preciso.

1.2 O que é preciso para verificar

Model checking precisa de três coisas, e a qualidade da resposta depende das três:

1. **Um modelo formal** do sistema — aqui, a Rede de Petri Colorida lida do arquivo `.cpn`;
2. **Uma propriedade formal**, escrita numa lógica com significado matemático preciso — aqui, CTL;

3. **Um algoritmo** que decide se o modelo satisfaz a propriedade, e que, quando não satisfaz, exibe *por quê*.

O terceiro ponto é o que separa model checking de uma prova de teorema comum: quando a resposta é "falso", a ferramenta devolve um **contraexemplo** — uma execução concreta que viola a propriedade. É a diferença entre um médico dizer "algo está errado" e apontar exatamente onde.

Capítulo 2

Model checking do começo

2.1 O modelo: sistemas de transição

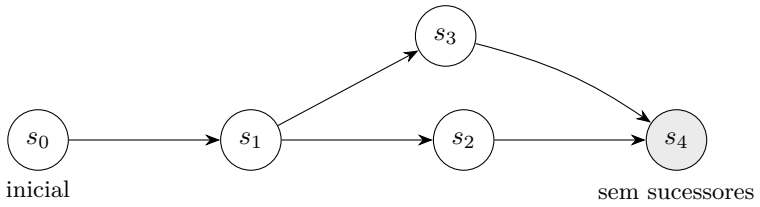
Tudo em model checking parte de um objeto simples. Um **sistema de transição** (ou *estrutura de Kripke*) é uma quádrupla

$$K = (S,\ s_0,\ R,\ L)$$

onde:

- S é um conjunto de **estados**;
- $s_0 \in S$ é o **estado inicial**;
- $R \subseteq S \times S$ é a **relação de transição**: $(s, s') \in R$ significa "de s o sistema pode ir para s'";
- L rotula cada estado com as **proposições atômicas** que valem nele — fatos elementares como "o place *Prediction* tem token".

Um **caminho** é uma sequência $s_0 \to s_1 \to s_2 \to \cdots$ seguindo R. Um caminho é **maximal** quando não pode ser estendido: ou é infinito, ou termina num estado sem sucessores.



Nesse exemplo há dois caminhos maximais: $s_0 s_1 s_2 s_4$ e $s_0 s_1 s_3 s_4$. Ambos terminam — s_4 é o que chamaremos de *marcação morta*.

2.2 A propriedade: lógica temporal CTL

CTL (*Computation Tree Logic*) descreve como proposições se comportam *ao longo do tempo* e *ao longo dos caminhos possíveis*. Cada operador CTL é a combinação de duas escolhas:

Quantificador de caminho	Operador temporal
A — em *todos* os caminhos	**X** — no próximo estado
E — em *algum* caminho	**F** — em *algum* estado futuro
	G — em *todos* os estados futuros
	U — até que

Combinando, obtêm-se os operadores usados na prática. Esta é a tabela mais importante do documento:

Fórmula	Nome	Leitura
$AG\,\varphi$	invariante, segurança	φ vale em *toda* marcação alcançável. "Nada de ruim acontece."
$AF\,\varphi$	inevitabilidade, vivacidade	*toda* execução maximal chega a um estado onde φ vale. "Algo de bom acontece, sempre."
$EF\,\varphi$	alcançabilidade	*existe* execução que chega a φ. "É possível chegar lá."
$E[\varphi\ U\ \psi]$	até que (existencial)	existe execução em que φ vale continuamente até que ψ passe a valer.

Por que essas quatro bastam

As demais se derivam. Em particular:

$$\neg EF\,\varphi \;\equiv\; AG\,\neg\varphi, \qquad \neg E[\varphi\ U\ \psi] \;=\; \text{“}\psi \text{ nunca acontece antes de } \varphi\text{”}$$

A segunda é a forma das propriedades de **precedência**: "o estágio m nunca é concluído antes do estágio $m-1$". Guarde essa leitura; ela reaparece no catálogo (propriedades C1 e C2).

Segurança e vivacidade são diferentes na prática

A distinção não é acadêmica, e ela determina como é o contraexemplo:

- Um **invariante** (AG) é violado por *um único estado ruim*. O contraexemplo é finito: a sequência mais curta do início até esse estado.
- Uma **vivacidade** (AF) é violada por *uma execução inteira* que nunca chega ao objetivo. Se essa execução for infinita, *não existe contraexemplo finito* — a menos que se exiba a parte que se repete. Essa é a forma *laço* (*lasso*), e o capítulo 6 mostra como o CPNCheck a produz.

2.3 O algoritmo: pontos fixos

CTL é decidida sem enumerar caminhos — o que seria inviável, já que há infinitos. Em vez disso, calcula-se para cada fórmula φ o **conjunto de satisfação** $\mathrm{Sat}(\varphi) \subseteq S$: os estados onde ela vale. A resposta final é simplesmente $s_0 \in \mathrm{Sat}(\varphi)$.

Os conjuntos são calculados por **pontos fixos**, de dentro para fora:

EF e $E[U]$: alcançabilidade para trás. $\mathrm{Sat}(EF\,\varphi)$ é o menor conjunto que contém $\mathrm{Sat}(\varphi)$ e é fechado por predecessores. Algoritmo: parta dos estados onde φ vale e caminhe *para trás* pelas arestas até saturar.

***AF*: "todo sucessor já está".** $\mathrm{Sat}(AF\,\varphi)$ é o menor conjunto Z tal que $Z = \mathrm{Sat}(\varphi) \cup \{s : \text{todo sucessor de } s \text{ está em } Z\}$. Implementa-se com um contador por estado: quando o último sucessor de s entra no conjunto, s entra também.

***AG*: o complemento de uma alcançabilidade.** $AG\,\varphi$ vale em s exatamente quando *nenhum* estado alcançável a partir de s viola φ. Então: calcule os estados ruins, calcule quem alcança um estado ruim (para trás), e tome o complemento.

Custo. Cada operador custa uma varredura do grafo, então o total é $O(|\varphi| \times (|S| + |R|))$ — *linear* no tamanho do grafo. O gargalo nunca é a lógica; é o tamanho de S.

2.4 O ponto delicado: marcações mortas

CTL de livro-texto assume que a relação R é **total**: todo estado tem ao menos um sucessor, e portanto todo caminho é infinito. Mas os modelos aqui são feitos para *parar*: a rede classifica a entrada e termina numa **marcação morta** (nenhuma transição habilitada).

Isso obriga a decidir o que $AF\,\varphi$ significa num caminho que acaba. Existem duas convenções, e elas dão respostas *diferentes*:

Convenção	O que diz numa marcação morta sem φ
livro-texto	A marcação morta ganha um laço sobre si mesma (Baier & Katoen, seção 3.1). O caminho vira infinito, nunca alcança φ, e portanto **falsifica** $AF\,\varphi$.
ASK-CTL (CPN Tools)	O `EV` do ASK-CTL é *vacuamente verdadeiro* numa marcação sem sucessores. Assim `EV(`φ`)` significa na verdade $AF(\varphi \vee \mathit{morta})$: uma rede que trava antes de decidir **satisfaz** `EV(PRED)`.

A diferença aparece exatamente quando existe uma marcação morta alcançável sem φ — ou seja, no caso em que você mais quer ser avisado.

Como o CPNCheck resolve. Ele adota a leitura de livro-texto (mais severa) e, ao gerar o script para o CPN Tools, codifica $AF\,\varphi$ como

$$\texttt{EV}(\varphi) \;\wedge\; \neg\,\texttt{E}[\neg\varphi\ \texttt{U}\ (\mathit{morta} \wedge \neg\varphi)]$$

para que o CPN Tools responda à *mesma* pergunta. Além disso, reporta as duas leituras (chaves `A3_ev`, `B2_ev`, `C3a_ev`, `D2a_ev`), de modo que a diferença fique visível em vez de escondida.

2.5 O limite real: explosão de estados

O número de estados cresce exponencialmente com o paralelismo do modelo. Para os modelos de *gradient boosting* gerados pelo pyRuleAnalyzer, com K classes e M estágios por classe, o grafo de ocorrências tem exatamente

$$(3M+3)^K + 2 \text{ nós} \qquad \text{e} \qquad K(3M+2)(3M+3)^{K-1} + 2 \text{ arcos},$$

porque os canais de classe intercalam livremente. Um modelo de três classes com cem estágios está fora do alcance de qualquer ferramenta explícita — inclusive o CPN Tools.

Isso não é defeito da implementação; é a natureza do problema. O que uma ferramenta honesta faz é **recusar-se a responder** quando não terminou a exploração, e é o que o CPNCheck faz: acima do orçamento (`max_nodes`, 200.000 por padrão) as propriedades de espaço de estados ficam *sem veredicto*, enquanto a análise estrutural e o teste de conformidade continuam valendo.

Capítulo 3

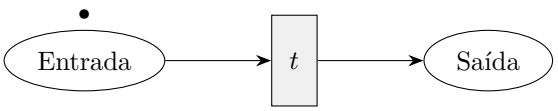
Redes de Petri Coloridas

3.1 Rede de Petri

Uma Rede de Petri tem dois tipos de nó:

- **Lugares** (*places*), desenhados como elipses, que guardam **fichas** (*tokens*);
- **Transições** (*transitions*), desenhadas como retângulos, que consomem fichas de alguns lugares e produzem em outros.

Os **arcos** ligam lugares a transições e vice-versa. O estado do sistema é a **marcação**: quantas fichas há em cada lugar. Uma transição está **habilitada** quando há fichas suficientes nos lugares de entrada; quando **dispara**, consome-as e produz as de saída.



guarda: `[#f0 x <= 3.0]`

3.2 O que "colorida" acrescenta

Numa Rede de Petri comum as fichas são indistinguíveis. Numa **Rede de Petri Colorida** (CPN) cada ficha carrega um **valor** — uma "cor". Isso traz quatro elementos:

Conjuntos de cores (*colour sets*). O tipo dos valores: `INT`, `REAL`, `BOOL`, produtos, registros, listas. No modelo de uma floresta, por exemplo, `SAMPLE` é um registro de seis reais.

Variáveis. Declaradas com um conjunto de cores: `var x: SAMPLE;`

Inscrições de arco. Expressões em CPN ML. No arco de entrada é um *padrão* que casa com a ficha; no de saída é uma expressão que constrói a ficha produzida.

Guardas. Condições booleanas na transição, como `[#f0 x <= 3.0 andalso #f1 x > 1.5]`.

Elementos de ligação

Aqui está o conceito que mais confunde e que o CPNCheck trata explicitamente. Uma transição por si só não é uma ação: uma ação é um **elemento de ligação** (*binding element*) — a transição *mais* uma atribuição de valores às suas variáveis.

As variáveis ganham valor de dois modos:

1. **Por casamento de padrão**: a variável aparece num arco de entrada, e seu valor vem da ficha consumida.
2. **Por enumeração sobre o conjunto de cores**: a variável aparece só na guarda ou num arco de saída. Nesse caso o CPN Tools percorre *todos* os valores do conjunto de cores dela.

O segundo caso é fácil de esquecer e caro de ignorar: uma ferramenta que não o implementa está analisando uma rede *menor* do que a desenhada — e não avisa. O CPNCheck implementa a enumeração e, quando o conjunto de cores é infinito (um `REAL`, por exemplo), **recusa a rede nomeando a variável** em vez de seguir em frente.

3.3 Hierarquia

Redes grandes se organizam em **páginas**. Uma *transição de substituição* não dispara: ela representa uma subpágina inteira. Os lugares da página de cima (*sockets*) são fundidos com os lugares de borda da subpágina (*ports*).

Nos modelos gerados pelo pyRuleAnalyzer: a página de topo tem `Input` e `Prediction`; cada árvore ou canal de classe é uma subpágina.

Para verificar, o CPNCheck **achata** a hierarquia: funde cada par port/socket num único lugar e produz uma rede plana equivalente. Nada se perde — a hierarquia é organização visual, não semântica.

3.4 O grafo de ocorrências

Aqui os dois capítulos se encontram. O **grafo de ocorrências** (*occurrence graph*, ou espaço de estados) de uma CPN é construído assim:

- cada **nó** é uma marcação alcançável;
- cada **arco** é a ocorrência de um elemento de ligação.

Esse grafo é exatamente a estrutura de Kripke do capítulo 2. As marcações são os estados S, as ocorrências são a relação R, a marcação inicial é s_0, e as proposições atômicas são fatos sobre as fichas ("o lugar *Prediction* tem ao menos uma ficha"). Por isso CTL se aplica diretamente.

Uma sutileza importante: as marcações carregam os **valores reais** das fichas. Não há abstração. Duas marcações que diferem no valor de uma única ficha são *dois nós distintos*. Isso torna a verificação exata para a entrada dada, e é também por isso que a verificação é feita *por entrada* — ponto ao qual voltaremos na seção 4.1.

3.5 Três conceitos derivados do grafo

Marcação morta. Nó sem sucessores: a rede parou. Numa rede correta, toda marcação morta contém uma predição.

Componente fortemente conexa (SCC). Conjunto maximal de nós mutuamente alcançáveis. A decomposição em SCCs (algoritmo de Tarjan) revela ciclos.

Home marking. Marcação alcançável a partir de *qualquer* marcação alcançável. Existe exatamente quando o grafo de SCCs tem uma *única* componente terminal (Jensen & Kristensen). É o que o `ListHomeMarkings` do CPN Tools calcula.

Capítulo 4

Para que serve isso num modelo de *machine learning*

4.1 O que se está verificando — e o que não se está

Vale ser muito claro, porque é aqui que se cometem exageros.

O CPNCheck verifica a **rede exportada**, lida de volta do arquivo `.cpn`. Ele **não** verifica que o modelo de ML é acurado, justo, robusto a ataque adversarial ou adequado ao problema. Ele verifica que o *artefato formal* que você vai publicar, embarcar ou citar se comporta como um classificador deve se comportar — e que ele computa a mesma coisa que o classificador de origem nas entradas testadas.

Dito isso, as garantias obtidas são reais e não-triviais:

- **Determinismo.** Duas folhas da mesma árvore nunca estão habilitadas simultaneamente — a árvore não é ambígua.
- **Terminação.** A rede sempre para; não existe ciclo alcançável.
- **Ausência de travamento.** Toda parada é uma parada *com resposta*.
- **Saída única e válida.** Nunca duas predições; nunca um rótulo fora do domínio declarado.
- **Disjunção das guardas.** As folhas de uma árvore particionam o espaço de entrada — e esta vale para *toda* entrada, não só as exploradas.
- **Conformidade.** Nas entradas testadas, a rede prediz o mesmo que o classificador.

4.2 A escada de escopo

As quatro técnicas do CPNCheck têm alcances *diferentes*, e confundir isso é o erro mais fácil de cometer ao relatar resultados:

Técnica	**Escopo**	**Um veredicto “verdadeiro” significa**
`ctl`	uma entrada	Para *esta* marcação inicial, a propriedade vale em todos os comportamentos possíveis. Outra entrada é outro grafo.
`scc`	uma entrada	Idem; é uma pergunta sobre a estrutura do mesmo grafo.
`structural`	**toda** entrada	Vale para o espaço de entrada inteiro, sem explorar nada. É o veredicto mais forte que a ferramenta produz.
`testing`	as entradas dadas	Equivalência funcional nas entradas fornecidas. É teste, não prova.

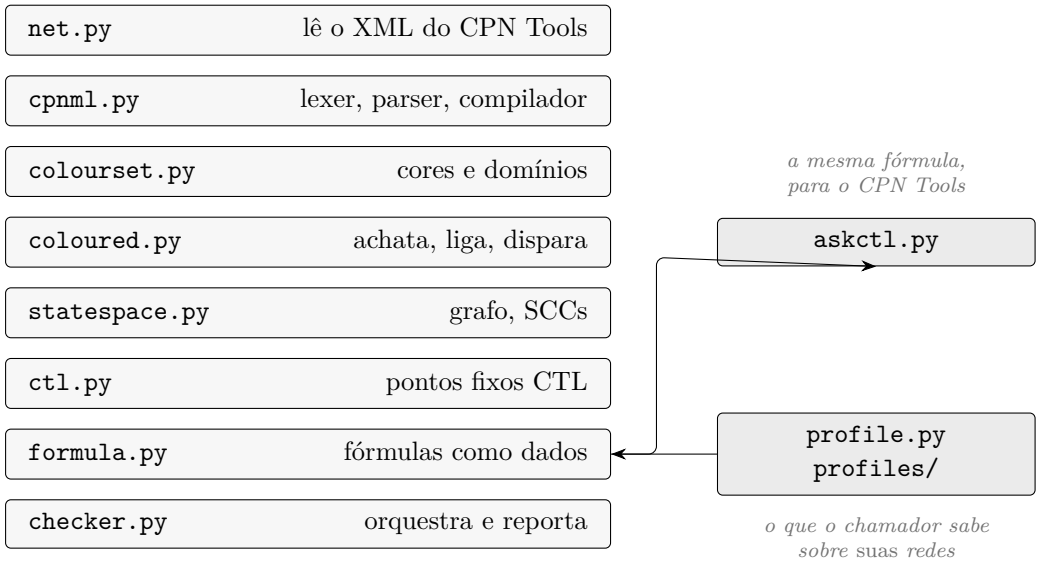
Como relatar sem exagerar. “As propriedades A1–A8 e B1–B3 foram verificadas para n entradas do conjunto de teste; B4 (disjunção de guardas) vale para todo o espaço de entrada; PC foi testada em N amostras.” Isso é exato. Dizer “o modelo foi formalmente verificado” sem qualificação não é.

Capítulo 5

CPNCheck por dentro

5.1 Arquitetura

O CPNCheck tem cerca de 9.000 linhas de Python e **nenhuma dependência de terceiros** — só a biblioteca padrão. As camadas vão do arquivo até o veredicto:



5.2 Camada por camada

5.2.1 `net.py` — ler o arquivo

Faz *só* o *parsing* do XML que o CPN Tools grava: páginas, lugares com suas marcações iniciais, transições com guardas, arcos com inscrições, transições de substituição com seus pares port/socket.

São 96 linhas, e não sabem nada sobre o que a rede significa. Isso é deliberado: a mesma leitura serve para qualquer rede.

5.2.2 `cpnml.py` — entender as expressões

Guardas e inscrições são escritas em **CPN ML**, um subconjunto de Standard ML. Uma versão anterior as traduzia com expressões regulares — o que funciona para o que um gerador emite e falha para o que uma pessoa escreve.

Hoje é um *parser* de verdade: analisador léxico, analisador sintático por escalada de precedência, e um compilador que transforma a árvore sintática em *closures* Python. Cobre:

expressões	`if/then/else`, `let/in/end`, `case/of`, `fn`, aplicação de função
operadores	a tabela do SML com precedência e associatividade: `orelse`, `andalso`, comparações, `::`, `@`, `+ - ^`, `* / div mod`, `~` e `not` unários
valores	inteiros, reais (inclusive `~` e `E~`), strings, booleanos, unit, tuplas, listas, registros, *multisets*
padrões	variáveis, curinga, literais, tuplas, listas, `::`, registros, anotações de tipo
declarações	`val` e `fun`, incluindo funções multi-cláusula e recursão

Consequência que vale destacar: a rede se auto-descreve. As funções `vadd`, `vsum`, `argmax`, `allzero` e `decide` usadas pelos modelos de floresta *não* são funções Python embutidas no verificador. Elas são lidas dos blocos `<ml>` do próprio arquivo `.cpn`, compiladas e executadas. Nada sobre votação suave ou *boosting* está dentro da ferramenta.

A compilação para *closures* (em vez de interpretação nó a nó) mantém o disparo rápido: a árvore é percorrida uma única vez, na compilação.

5.2.3 `colourset.py` — os domínios

Lê os conjuntos de cores e as variáveis dos elementos XML estruturados (`<color>`, `<var>`) e decide quais são *finitos*:

Finitos	`unit`, `bool`, enumerações, faixas de índice, inteiros com limites, e produtos/registros/subconjuntos/*aliases* construídos a partir desses
Infinitos	`int` sem limites, `real`, `string`, listas

Um domínio infinito é reportado como "desconhecido", nunca chutado.

5.2.4 `coloured.py` — a rede que dispara

Achata a hierarquia (fundindo ports e sockets), compila cada guarda e inscrição, e implementa as duas operações centrais:

`enabled(marcação)` enumera os elementos de ligação habilitados: casa os padrões dos arcos de entrada contra as fichas disponíveis e, em seguida, percorre os domínios das variáveis livres.

`fire(marcação, transição, ligação)` calcula a marcação sucessora: remove as fichas consumidas, avalia as inscrições de saída, deposita as produzidas.

As fichas são valores Python escolhidos para serem *hasheáveis e ordenáveis*: registro é tupla na ordem dos campos declarados, lista é tupla, *unit* é a tupla vazia. Assim duas marcações são o mesmo nó exatamente quando são iguais — o que é o que permite usar um dicionário para reconhecer marcações já visitadas.

5.2.5 `statespace.py` — o grafo

Busca em largura a partir da marcação inicial, com orçamento de nós. Para cada nó guarda o *pai* na árvore de busca — incluindo a **ligação** usada, sem a qual um traço não poderia ser redisparado. Em seguida, decomposição em SCCs por Tarjan iterativo, de onde saem as componentes terminais e os *home markings*.

5.2.6 `ctl.py` — os pontos fixos

Três funções, exatamente as do capítulo 2:

`_eu`	$E[\varphi\ U\ \psi]$ por alcançabilidade para trás
`_af`	$AF\,\varphi$ por contagem de sucessores, com o parâmetro `semantics` escolhendo entre a leitura de livro-texto e a do ASK-CTL (seção 2.4)
`_af_lasso`	extrai o *laço* quando AF falha numa execução infinita (capítulo 6)

5.2.7 `formula.py` — propriedades como dados

Este é o módulo que dá flexibilidade à ferramenta. Uma propriedade não é código Python escrito contra uma rede específica; é uma *árvore de objetos*:

Proposições atômicas	`Dead`, `Marked`, `AllMarked`, `MaxTokens`, `TokensIn`, `Enabled`, `EnabledCount`
Conectivos	`Not`, `And`, `Or`, `Implies`
Temporais	`AG`, `AF`, `EF`, `EU`
Quantificador	`ForEach` — “para toda árvore...”, “para todo canal...”

Cada fórmula faz duas coisas além de ser verdadeira ou falsa:

1. **explica-se** quando falha — o `AG` devolve a sequência mais curta até a marcação violadora e pergunta ao operando *por que aquela marcação é ruim*;
2. **renderiza-se em ASK-CTL** — a *mesma* árvore vira a consulta para o CPN Tools.

Por que isso importa para a confiança. Antes, a verificação em Python e o script ASK-CTL eram escritos à mão, lado a lado. Nada além de cuidado impedia que divergissem — e uma divergência silenciosa significaria comparar respostas para perguntas diferentes. Hoje há uma única árvore e dois *backends*, e a divergência é impossível por construção.

5.2.8 `profile.py` e `profiles/` — onde entra o domínio

O verificador não sabe que existe um lugar chamado `Prediction`. Quem sabe é o **perfil**, que responde:

`matches` / `family`	esta rede é uma das minhas, e como eu a chamo
`resolve_groups`	os domínios que o `ForEach` percorre
`ctl_formulas`	as propriedades temporais, como fórmulas
`scc_properties`	as respondidas pela decomposição em SCCs
`static_checks`	vereditos que dispensam grafo de ocorrências
`conformance`	comparação com uma implementação de referência
`label_domain`	os valores de saída admissíveis
`as_input_token` / `input_places`	como outra entrada entra na rede
`mutation_operators`	as falhas que as propriedades devem pegar
`unavailable`	propriedades que a rede tem mas esta execução não decide
`catalog`	os descritores sob os quais os vereditos são reportados

O perfil `ml_trees` é o exemplo completo: ele reconhece as redes do pyRuleAnalyzer e declara as vinte propriedades do catálogo.

A distinção entre "não se aplica" e "não pôde ser decidida" foi um defeito real encontrado tarde. Uma propriedade que a rede simplesmente não tem (agregação numa rede que não agrega) é omitida e nunca mencionada — certo. Mas uma propriedade que a rede *tem* e que não pôde ser decidida (o domínio de rótulos de uma árvore de decisão, que não está escrito em lugar algum legível) estava sendo omitida também — o que se lê como se tivesse passado. Hoje ela é reportada como `SKIP` com o motivo.

5.3 As quatro técnicas

`ctl`	Model checking explícito sobre o grafo de ocorrências. Valores reais de ficha, sem abstração.
`scc`	*Home markings* pela decomposição em componentes fortemente conexas. Não é uma fórmula CTL.
`structural`	Análise das guardas que são conjunções de limites. Uma guarda `#f0 x <= 3.0 andalso #f1 x > 1.5` descreve uma *caixa*; duas guardas podem valer juntas exatamente quando as caixas se intersectam. Vale para *toda* entrada.
`testing`	Conformidade com uma implementação de referência nas entradas dadas. Segue uma sequência maximal por entrada, o que é barato o bastante para um conjunto de teste inteiro.

A técnica `testing` só é significativa *ao lado* do model checking, nunca no lugar dele: ela segue *uma* execução maximal por entrada, e que essa execução dê "o" resultado da rede depende de confluência — que é justamente o que as propriedades de determinismo estabelecem.

5.4 Exatidão da análise estrutural

Um detalhe que mudou durante o desenvolvimento e que vale registrar, porque é um caso de resposta errada dada em silêncio.

A versão original tratava os extremos fechados como se fossem abertos, mapeando $\geq$ em $>$ e $<$ em $\leq$, e testava a sobreposição com max(inferiores) $<$ min(superiores).

Considere então as guardas $x \geq 3$ e $x \leq 3$, que compartilham o ponto $x = 3$. O teste dava $3 < 3$, ou seja *disjuntas* — um **falso negativo**, que é exatamente o erro que importa: dizer "não há sobreposição" quando há.

A versão atual rastreia se cada extremo é aberto ou fechado e decide exatamente:

$y \geq 3$	*vs* $y \leq 3$	sobrepõem em $\{3\}$	✓
$y > 3$	*vs* $y \leq 3$	disjuntas	✓
$y \geq 3$	*vs* $y < 3$	disjuntas	✓

Nenhum veredicto sobre as redes geradas mudou — o exportador só emite `<=` e `>`, onde as duas versões concordam. Mas para uma rede escrita à mão era uma resposta errada esperando para acontecer.

5.5 O catálogo de propriedades

As vinte propriedades das redes do pyRuleAnalyzer. Note a coluna *Técnica*: ela determina o escopo do veredicto, conforme a seção 4.1.

ID	Propriedade	Fórmula	Técnica
Comuns às três famílias			
A1	Terminação	$AF\ morta$	ctl
A2	Sem travamento espúrio	$\neg EF(morta \wedge \neg pred)$	ctl
A3	Decisão inevitável	$AF\ pred$	ctl
A4	Predição recuperável	$AG\ EF\ pred$	ctl
A5	Saída única	$AG\ \lvert Prediction\rvert \leq 1$	ctl
A6	Rótulo válido	$AG\ Prediction \subseteq L$	ctl
A7	Segurança (*safeness*)	$AG\ \forall p:\ \lvert p\rvert \leq 1$	ctl
A8	*Home marking*	uma única componente terminal	scc
Por árvore			
B1	Determinismo de folha	$\forall T:\ AG\ \lvert EN(folhas_T)\rvert \leq 1$	ctl
B2	Seleção de folha inevitável	$\forall T:\ AF\ EN(folhas_T)$	ctl
B3	Saída única por árvore	$\forall T:\ AG\ \lvert out_T\rvert \leq 1$	ctl
B4	Disjunção das guardas	$\forall T, i \neq j:\ caixa_i \cap caixa_j = \emptyset$	**structural**
Gradient boosting			
C1	Precedência de estágio	$\forall m:\ \neg E[\neg acc_{m-1}\ U\ acc_m]$	ctl
C2	Sem escore prematuro	$\neg E[\neg acc_M\ U\ score]$	ctl
C3a	Disparo de decisão inevitável	$AF\ EN(decide)$	ctl
C3b	Determinismo da decisão	$AG\ \lvert EN(decide)\rvert \leq 1$	ctl
Floresta aleatória			
D1	Voto completo	$AG(EN(vote) \rightarrow todos\ votaram)$	ctl
D2a	Agregação inevitável	$AF\ EN(vote)$	ctl
D2b	Agregação única	$AG\ \lvert Prediction\rvert \leq 1$	ctl
Conformidade			
PC	Consistência da predição	$rede(x) = modelo(x)$	**testing**

onde $EN(G)$ vale numa marcação com arco de saída cuja transição está em G, e $|EN(G)|$ conta esses arcos. As propriedades C1 e C2 são quantificadas também sobre os canais de classe k, omitido acima por brevidade: acc_m é o acumulador do estágio m do canal, e M é o último estágio.

Capítulo 6

Contraexemplos

6.1 Por que um veredicto sozinho não basta

"A propriedade A3 é falsa" é uma informação pobre. Ela não diz se o problema é do modelo, da exportação ou da propriedade; não permite reproduzir; não permite corrigir. Um model checker sério devolve a **evidência**.

No CPNCheck o objeto `Counterexample` carrega a sequência de ocorrências, a **ligação de cada passo**, a marcação final e a **entrada** que a violação exige.

6.2 As cinco formas

`finite`	Uma propriedade de segurança falhou: existe marcação alcançável que a viola, e o prefixo é a sequência mais curta até ela.
`lasso`	Uma propriedade de vivacidade falhou numa execução infinita: um prefixo até um ciclo que a rede pode repetir para sempre sem nunca chegar ao objetivo. **Sem o ciclo, este caso não tem testemunha finita alguma.**
`witness`	Uma falha que não é uma sequência — por exemplo, duas componentes terminais sem sucessor comum.
`structural`	Um defeito da rede como escrita, independente de execução: duas guardas que podem valer juntas. Carrega um *ponto* do espaço de entrada onde isso acontece.
`conformance`	A rede e a referência discordam numa entrada concreta.

6.3 O laço, e por que ele precisou de um algoritmo

Quando $AF\,\varphi$ falha, há dois motivos possíveis:

1. a execução *para* antes de φ — e então o contraexemplo é um caminho finito até uma marcação morta fora de φ;
2. a execução *continua para sempre* sem alcançar φ.

No segundo caso não existe caminho finito que sirva de prova. A versão anterior da ferramenta apenas informava "existe um ciclo alcançável que evita ...", sem nada para examinar.

O algoritmo `_af_lasso` extrai o ciclo, e é correto por construção. O argumento:

Seja S o conjunto dos nós que *não* satisfazem $AF\,\varphi$. Nenhum nó de S satisfaz φ. Além disso, todo nó de S que tem sucessores tem *um sucessor em* S — pois se todos os sucessores satisfizessem $AF\,\varphi$, o próprio nó satisfaria. Logo, uma caminhada a partir da marcação inicial dentro de S só pode parar numa marcação morta. Quando nenhuma marcação morta de S é alcançável — que é exatamente o caso em que a busca por caminho finito não achou nada — a caminhada **tem** de repetir um nó, e a repetição é o ciclo.

Na prática o traço sai assim:

```
A3  [lasso] a predicao nunca acontece
    sequence: Distribute<x = (-1.36, -1.91, ...)>
              -> Init<x = (...)>
              -> Feed_1<sc = -0.043, x = (...)>
              then forever [GBDT1T1_Rule2_Class1<x = (...)>]
    cycle_length: 1
```

6.4 O certificado: `replay()`

Esta é a parte que sustenta a confiança na ferramenta.

Um contraexemplo é uma sequência finita de elementos de ligação. O método `replay()` **redispara** essa sequência a partir da marcação inicial usando a *regra de disparo da rede* — não a busca que produziu o traço — e confere que ela termina onde o contraexemplo afirma. Para um laço, confere também que o ciclo retorna ao ponto de entrada.

Por que isso é forte. A busca (pontos fixos, BFS, Tarjan) e a regra de disparo são código independente. Um erro nos pontos fixos *não consegue forjar* um traço que redispara. Um contraexemplo que passa no `replay()` é um contraexemplo que a ferramenta não inventou.
A suíte de testes exige que *toda* sequência de *todo* mutante redispare — e exige que uma sequência adulterada (invertida ou truncada) **não** redispare. Sem esse segundo teste, o certificado não certificaria nada.

6.5 De achado estrutural para entrada concreta

A disjunção de guardas (B4) vale ou falha para *toda* entrada, então uma falha não nomeia execução nenhuma. Mas a interseção de duas guardas em caixa é uma caixa, e um ponto dela é uma **entrada** onde a sobreposição de fato acontece.

O CPNCheck calcula esse ponto e o devolve como `cex.input`. Realimentando-o:

```
resultado = check_cpn("modelo.cpn", class_labels=[0, 1])
# B4 FAIL  |  B1 PASS   (a entrada original nao alcanca o par)

x = resultado.properties["B4"].cex.input     # (1.31, 0, 0, 1.117, 0, 0)

de_novo = check_cpn("modelo.cpn", samples=[x], class_labels=[0, 1])
# B4 FAIL  |  B1 FAIL   -- e no MESMO par de folhas
```

Isso fecha o ciclo entre a prova estrutural (toda entrada) e a prova de estado (uma entrada): o achado deixa de ser uma afirmação e passa a ser *reprodutível*. É a ideia de refinamento guiado por contraexemplo (CEGIS) aplicada aqui.

Capítulo 7

Por que confiar no resultado

7.1 O problema de fundo

Um verificador é um programa, e programas têm defeitos. Como saber que um "VERIFIED" não é apenas um *bug* que responde "verdadeiro" para tudo?

Duas evidências independentes sustentam a resposta, e é importante que sejam *duas*, porque cada uma sozinha é insuficiente.

7.2 Primeira evidência: sensibilidade (mutantes)

De uma rede correta, o CPNCheck deriva **dezesseis** redes defeituosas — cada uma uma pequena edição de guarda, inscrição ou arco, escolhida para quebrar *exatamente* uma propriedade nomeada:

Mutante	Deve falhar	Edição
`cycle`	A1	folha viva devolve a ficha de entrada
`deadlock`	A2, A3	produtor da predição perde o arco de saída
`dead_branch`	A4, A8	folha dormente sempre habilitada, sem produzir
`double_output`	A5, A7	predição depositada duas vezes
`bad_label`	A6	predição substituída pelo rótulo 7
`overlap`	B1, B4	folha dormente habilitada junto com a viva
`hidden_overlap`	B4	duas folhas dormentes com a mesma guarda
`no_leaf`	B2	guarda da folha viva vira falsa
`tree_double`	B3	árvore emite o resultado duas vezes
`skip_stage`	C1	`Feed_2` lê `Acc0`
`early_score`	C2	`Finalize` lê $\texttt{Acc}_{M-1}$
`no_decision`	C3a	guarda do `Decide` vivo vira falsa
`two_decisions`	C3b	guarda de um `Decide` dormente vira verdadeira
`partial_vote`	D1	`Vote` deixa de consumir o voto da árvore 0
`no_vote`	D2a	guarda do `Vote` vira falsa
`wrong_value`	PC	rede prediz outra classe válida

O par `overlap` / `hidden_overlap` merece atenção. O primeiro habilita uma folha dormente ao lado da viva: a entrada do arquivo alcança as duas, e tanto B1 (por entrada) quanto B4 (estrutural) falham. O segundo dá a mesma guarda a duas folhas que a entrada *nunca alcança*: B1 **passa**, e só B4 falha. Esse par demonstra que as duas técnicas têm escopos genuinamente diferentes — não é retórica.

Do mesmo modo, `wrong_value` produz uma rede que satisfaz *todas* as propriedades CTL e falha apenas na conformidade: comportar-se bem e computar a coisa certa são perguntas distintas.

7.3 Segunda evidência: concordância com o CPN Tools

O CPNCheck executa o simulador do CPN Tools 4.0.1 de forma *headless* — através do Access/CPN, entrando na ferramenta de espaço de estados exatamente como o *Enter State Space* da interface faz, calculando os grafos de ocorrências e de SCCs e avaliando consultas ASK-CTL — e compara cada estatística e cada veredicto.

Redes comparadas (duas corretas + todos os mutantes de cada)	**28**
Redes em que *todos* os valores coincidem	**28**
Vereditos *falsos* confirmados pelo CPN Tools	**81**
Discordâncias	**0**

Por que os 81 são a parte que importa. Concordância apenas nas redes corretas também seria alcançada por dois verificadores que sempre respondem "verdadeiro". São os vereditos *falsos* — confirmados independentemente pelo CPN Tools — que mostram que ambos os motores detectam as mesmas violações nos mesmos lugares.

Essa comparação foi refeita *depois* de o motor de avaliação do CPN ML ser inteiramente substituído (expressões regulares → *parser* real): mesmas 28 redes, mesmos 81 vereditos falsos, nenhuma discordância. Nenhum veredicto se moveu.

7.4 Terceira evidência: os contraexemplos redisparam

Como descrito no capítulo 6, toda sequência produzida para todo mutante é redisparada com a regra de disparo da rede e tem de reproduzir a violação. É uma verificação da busca por código que não é a busca.

7.5 O que *não* está garantido

Esta seção é tão importante quanto as anteriores.

Limitação	**Consequência prática**
Verificação `ctl` é **por entrada**	Verificar 3 amostras não diz nada sobre a quarta. Só B4 (estrutural) vale para todo o espaço.
Explosão de estados	Acima de `max_nodes` as propriedades de espaço de estados ficam *sem veredicto* — e "sem veredicto" nunca deve ser lido como "aprovado". Quatro reduções opcionais adiam esse limite (conjuntos teimosos, *sweep-line*, equivalência e simetria); ver o documento sobre explosão de estados.
PC é teste	Conformidade nas entradas dadas, não prova de equivalência funcional.
O fragmento CPN ML tem limites	Tempo real (`@++`), valores aleatórios, canais síncronos e código com efeitos colaterais não são suportados (redes temporizadas com tempo inteiro, prioridades e conjuntos de fusão são). O que está fora levanta `UnsupportedNet` em vez de ser mal interpretado.
Variável livre sobre domínio infinito	A rede é **recusada**, nomeando a variável. Prosseguir verificaria uma rede menor do que a desenhada.
Verifica a rede, não o modelo	Acurácia, justiça e robustez do classificador são outras perguntas.

O princípio de projeto por trás de tudo isso: quando a ferramenta não sabe, ela *diz que não sabe*. Nunca chuta, nunca omite em silêncio. Três defeitos encontrados no desenvolvimento eram exatamente violações desse princípio — uma propriedade sumindo do relatório em vez de aparecer como `SKIP`; uma entrada sendo ignorada sem aviso; uma sobreposição de guardas reportada como disjunta. Os três foram corrigidos, e cada um virou teste.

Capítulo 8

Uso prático

8.1 Linha de comando

```
cpncheck check modelo.cpn --labels 0,1,2     # verifica e imprime o
    relatorio
cpncheck check modelo.cpn --trace            # + contraexemplos completos
cpncheck check modelo.cpn --json out.json    # vereditos em JSON
cpncheck check modelo.cpn --latex            # tabela LaTeX
cpncheck askctl modelo.cpn -o askctl.sml     # script para o CPN Tools
cpncheck mutants modelo.cpn mutantes/        # um mutante por propriedade
cpncheck oracle modelo.cpn                   # roda o CPN Tools e compara
```

8.2 Python

```
from cpncheck import check_cpn

resultado = check_cpn("modelo.cpn", class_labels=[0, 1, 2])
print(resultado.report())
assert resultado.passed

for r in resultado.failures:
    print(r.id, r.detail)          # o contraexemplo, renderizado
    cex = r.cex
    cex.kind                       # finite | lasso | witness | structural
        | conformance
    cex.sequence(bindings=True)    # com as ligacoes de cada passo
    cex.input                      # a entrada que a violacao exige
    cex.replay(checker.cnet)       # True: ele realmente faz o que diz
```

8.3 Como ler o relatório

```
Model checking: gbdt_binary.cpn  [Gradient Boosting Decision Trees]
======================================================================
  occurrence graph : 14 nodes, 13 arcs
  SCC graph        : 14 nodes, 13 arcs, 1 dead marking(s), 1 home marking(s)
  inputs           : 1 model checked, 0 tested   time: 0.00s
  semantics        : CTL over the occurrence graph, dead markings with
                     a self-loop

  CTL model checking on the occurrence graph
  ----------------------------------------------------------------------
```

```
A1   Termination                  AF dead                          PASS
...
16 passed, 0 failed, 0 skipped -- VERIFIED
```

Três coisas a observar:

1. A linha `inputs` diz *quantas entradas* foram verificadas. Um `VERIFIED` com `1 model checked` é uma afirmação sobre uma entrada.
2. A linha `semantics` declara a convenção de marcação morta em uso.
3. `skipped` não é `passed`. Uma propriedade pulada traz o motivo ao lado.

8.4 Verificando redes que não são de *machine learning*

As propriedades são dados, não código. Um perfil próprio permite verificar qualquer rede:

```python
import collections
from cpncheck import CPNModelChecker
from cpncheck.formula import AF, AG, Dead, MaxTokens
from cpncheck.profile import Profile
from cpncheck.selectors import Places

class MeuPerfil(Profile):
    name = "minhas redes"
    catalog = meus_descritores

    def matches(self, net):
        return any(p["name"] == "Done" for p in net.places)

    def ctl_formulas(self, net, groups, labels):
        return collections.OrderedDict([
            ("T1", AF(Dead().phrased("o fim"))),
            ("T2", AG(MaxTokens(Places(name="Done"), 1))),
        ])

print(CPNModelChecker("minha.cpn", profile=MeuPerfil()).check().report())
```

Glossário

Marcação O estado da rede: quantas fichas, e com que valores, há em cada lugar.

Elemento de ligação Uma transição *mais* uma atribuição de valores às suas variáveis. É a unidade de ação.

Grafo de ocorrências O grafo de todas as marcações alcançáveis; a estrutura de Kripke da rede.

Marcação morta Marcação sem transições habilitadas: a rede parou.

Home marking Marcação alcançável a partir de qualquer marcação alcançável.

SCC Componente fortemente conexa: conjunto maximal de nós mutuamente alcançáveis.

CTL Lógica temporal de árvores de computação; AG, AF, EF, $E[\cdot\, U\, \cdot]$.

Segurança / vivacidade "Nada de ruim acontece" (AG) *vs.* "algo de bom acontece" (AF).

Contraexemplo Evidência concreta de que uma propriedade não vale.

Laço (*lasso*) Contraexemplo com prefixo + ciclo, a forma necessária quando a violação é uma execução infinita.

Mutante Rede derivada de uma correta por uma edição que quebra uma propriedade nomeada; serve para provar que o verificador consegue responder "falso".

Perfil O objeto que informa ao verificador o que as redes de um domínio significam.

ASK-CTL A biblioteca de model checking do CPN Tools.

CPN ML O subconjunto de Standard ML usado em guardas e inscrições.